

Public-Key Cryptosystems

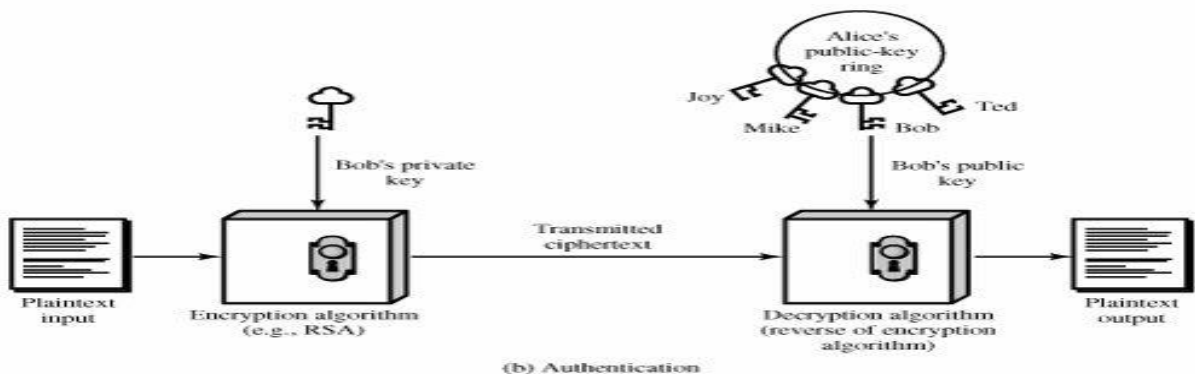
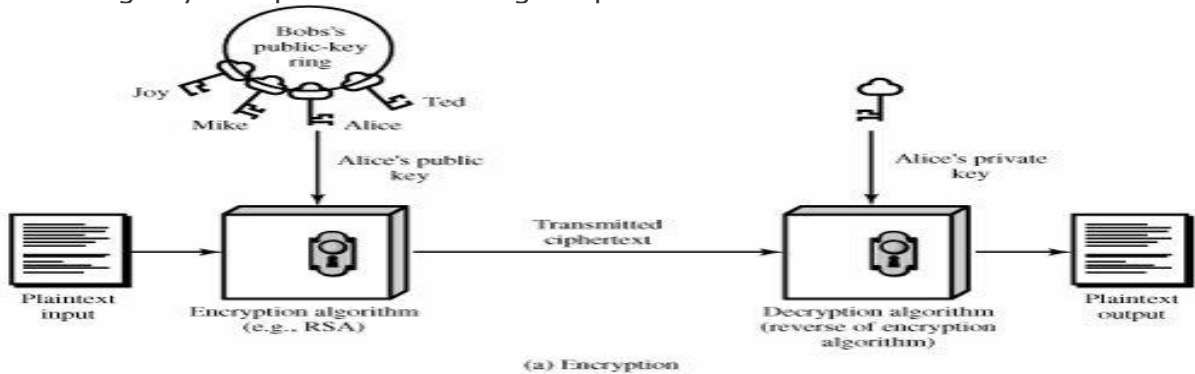
Asymmetric algorithms rely on one key for encryption and a different but related key for decryption.

These algorithms have the following important characteristics:

- It is computationally infeasible to determine the decryption key given only knowledge of the cryptographic algorithm and the encryption key.

A public-key encryption scheme has six ingredients

- **Plaintext:** This is the readable message or data that is fed into the algorithm as input.
- **Encryption algorithm:** The encryption algorithm performs various transformations on the plaintext.
- **Public and private keys:** This is a pair of keys that have been selected so that if one is used for encryption, the other is used for decryption. The exact transformations performed by the algorithm depend on the public or private key that is provided as input.
- **Ciphertext:** This is the scrambled message produced as output. It depends on the plaintext and the key. For a given message, two different keys will produce two different ciphertexts.
- **Decryption algorithm:** This algorithm accepts the ciphertext and the matching key and produces the original plaintext.



The essential steps are the following:

1. Each user generates a pair of keys to be used for the encryption and decryption of messages.
2. Each user places one of the two keys in a public register or other accessible file. This is the public key. The companion key is kept private. As [Figure 9.1a](#) suggests, each user maintains a collection of public keys obtained from others.
3. If Bob wishes to send a confidential message to Alice, Bob encrypts the message using Alice's public key.
4. When Alice receives the message, she decrypts it using her private key. No other recipient can decrypt the message because only Alice knows Alice's private key.

Applications of Public-Key crypto systems

Algorithm	Encryption/Decryption	Digital Signature	Key Exchange
RSA	Yes	Yes	Yes
Elliptic Curve	Yes	Yes	Yes
Diffie-Hellman	No	No	Yes
DSS	No	Yes	No

RSA Algorithm-

Let-

- Public key of the receiver = (e, n)
- Private key of the receiver = (d, n)

Then, RSA Algorithm works in the following steps-

Step-01:

At sender side,

- Sender represents the message to be sent as an integer between 0 and $n-1$.
- Sender encrypts the message using the public key of receiver.
- It raises the plain text message 'P' to the e th power modulo n .
- This converts the message into cipher text 'C'.
 $C = P^e \bmod n$

Step-02:

- The cipher text 'C' is sent to the receiver over the communication channel.

Step-03:

At receiver side,

- Receiver decrypts the cipher text using his private key.
- It raises the cipher text 'C' to the dth power modulo n.
- This converts the cipher text back into the plain text 'P'.

$$P = C^d \bmod n$$

Key Generation

Select p, q	p and q both prime, $p \neq q$
Calculate $n = p \times q$	
Calculate $\phi(n) = (p - 1)(q - 1)$	
Select integer e	$\gcd(\phi(n), e) = 1; 1 < e < \phi(n)$
Calculate d	$d \equiv e^{-1} \pmod{\phi(n)}$
Public key	$PU = \{e, n\}$
Private key	$PR = \{d, n\}$

Encryption

Plaintext:	$M < n$
Ciphertext:	$C = M^e \bmod n$

Decryption

Ciphertext:	C
Plaintext:	$M = C^d \bmod n$

Digital Signature:

Definition: A digital signature or digital signature scheme is a mathematical scheme for demonstration the authenticity of digital message or document.

Means, a digital signature is an authentication mechanism that enables the creator of a message to attach a code that act as a signature.

This signature is formed by taking the hash of the message and encrypting the message with the creator's private key. The signature guarantees the source and integrity of the message.

The digital signature standard (DSS) is an NIST standard that uses the secure hash algorithm (SHA).

Where it used:

Message authentication protects two parties who exchange message from any third party.

But it does not protect the two parties against each other.

Example: Suppose that john sends an authenticated message to marry, using any authentication scheme (symmetric or public key cryptography). There are two disputes (clash or fight or arguments) that could arise.

1. Mary may forge a different message & claim that it came from John. Means, Mary would simply have to create a message & append an authentication code using the key, which John and Mary share.
2. John can deny sending the message Because it is possible for mary to forge that john did n fact send the message.

Properties of Digital Signature:

- It must verify the author and the date and time of the signature.
- It must authenticate the contents at the time of the signature.
- It must be verifiable by third parties, to resolve disputes.

Digital Signature Requirements

1. The signature must be a bit pattern that depends on the message being signed.
2. The signature must use some information unique to the sender to prevent both forgery and denial.
3. It must be relatively easy to produce the digital signature.
4. It must be relatively easy to recognize and verify the digital signature.
5. It must be computationally infeasible to forge a digital signature, either by constructing a new message for an existing digital signature or by constructing a fraudulent digital signature for a given message.
6. It must be practical to retain a copy of the digital signature in storage.

Hash functions

Hash functions are used to create fixed-length digests of arbitrarily long input strings. Hash functions are keyless and provide data integrity. They are usually built using iterated and dedicated hash function construction techniques.

Various families of hash functions are available, such as MD, SHA-1, SHA-2, SHA-3, RIPEMD, and Whirlpool. Hash functions are commonly used for digital signatures and **Message Authentication Codes (MACs)**, such as HMACs. They have three security properties, namely preimage resistance, second preimage resistance, and collision resistance.

Compression of arbitrary messages into a fixed-length digest

This property relates to the fact that a hash function must be able to take an input text of any length and output a fixed-length compressed message. Hash functions produce a compressed output in various bit sizes, usually between 128-bits and 512-bits.

Easy to compute

Hash functions are efficient and fast one-way functions. It is required that hash functions be very quick to compute regardless of the message size. The efficiency may decrease if the message is too big, but the function should still be fast enough for practical use.

Preimage resistance

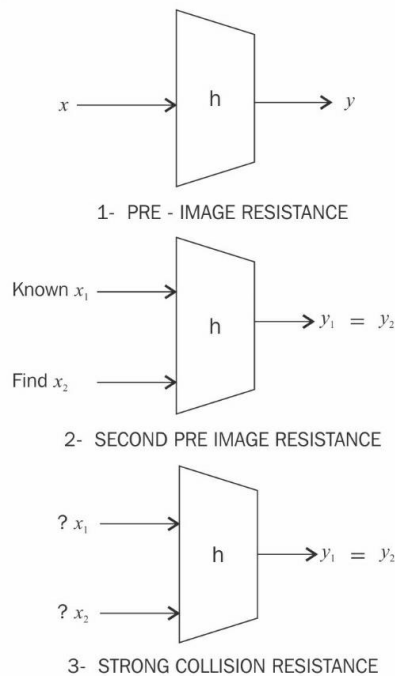
This property can be explained by using the simple equation shown as follows: $h(x) = y$. Here, h is the hash function, x is the input, and y is the hash. The first security property requires that y cannot be reverse-computed to x . x is considered a preimage of y , hence the name **preimage resistance**. This is also called a one-way property.

Second preimage resistance

The **second preimage resistance** property requires that given x and $h(x)$, it is almost impossible to find any other message m , where $m \neq x$ and $hash\ of\ m = hash\ of\ x$ or $h(m) = h(x)$. This property is also known as **weak collision resistance**.

The **collision resistance** property requires that two different input messages should not hash to the same output.

In other words, $h(x) \neq h(z)$. This property is also known as **strong collision resistance**. All these properties are shown in the following diagram:



Three security properties of hash functions

Due to their very nature, hash functions will always have some collisions. This is where two different messages hash to the same output. However, they should be computationally impractical to find. A concept known as the **avalanche effect** is desirable in all hash functions. The avalanche effect specifies that a small change, even a single character change in the input text, will result in an entirely different hash output.

Requirements for a hash function:

The purpose of a hash function is to produce a “fingerprint” of a file, message or other block of data. To be useful for message authentication, a hash function H must have the following properties:

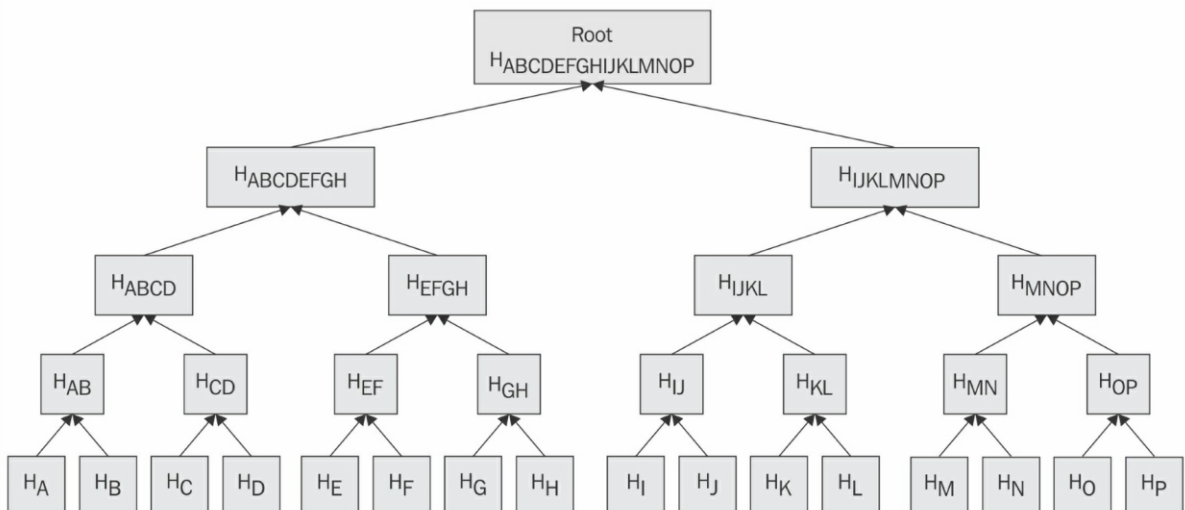
1. H Can be applied to a block of data of any size
2. H produces a fixed length output.
3. $H(x)$ is relatively easy to compute for any given x , making both hardware and software implementations practical.
4. One-way property:- for any given value h , it is computationally infeasible to find x such that $H(x)=h$. this sometimes referred to in the literature as the one way property.
5. Weak collision resistance:- for any given block x . it is computationally infeasible to find $y \neq x$ with $H(y)=H(x)$. this is referred as weak collision resistance.
6. Strong collision resistance:- it is computationally infeasible to find any pair (X,Y) such that $H(x)=H(y)$. this referred as strong collision resistance.

Requirements for a Cryptographic Hash Function H

Requirement	Description
Variable input size	H can be applied to a block of data of any size.
Fixed output size	H produces a fixed-length output.
Efficiency	$H(x)$ is relatively easy to compute for any given x , making both hardware and software implementations practical.
Preimage resistant (one-way property)	For any given hash value h , it is computationally infeasible to find y such that $H(y) = h$.
Second preimage resistant (weak collision resistant)	For any given block x , it is computationally infeasible to find $y \neq x$ with $H(y) = H(x)$.
Collision resistant (strong collision resistant)	It is computationally infeasible to find any pair (x, y) such that $H(x) = H(y)$.
Pseudorandomness	Output of H meets standard tests for pseudorandomness.

Merkle trees

The concept of Merkle tree was introduced by Ralph Merkle. A diagram of Merkle tree is shown here. **Merkle trees** enable secure and efficient verification of large datasets.



A Merkle tree

A Merkle tree is a binary tree in which the inputs are first placed at the leaves (node with no children), and then the values of pairs of child nodes are hashed together to produce a value for the parent node (internal node) until a single hash value known as **Merkle root** is achieved.

Elliptic Curve Digital Signature

Algorithm

In order to sign and verify using the ECDSA scheme, the first key pair needs to be generated:

1. First, define an elliptic curve E : With modulus P , Coefficients a and b

Generator point A that forms a cyclic group of prime order q

2. An integer d is chosen randomly so that $0 < d < q$.

3. Calculate public key B so that $B = d A$.

The public key is the sextuple in the form shown here: $Kpb = (p, a, b, q, A, B)$

The private key, d is randomly chosen in step 2: $Kpr = d$

Now the signature can be generated using the private and public key.

4. First, an ephemeral key Ke is chosen, where $0 < Ke < q$. It should be ensured that Ke is truly random and that no two signatures have the same key; otherwise, the private key can be calculated.

5. Another value R is calculated using $R = Ke A$; that is, by multiplying A (the generator point) and the random ephemeral key.

6. Initialize a variable r with the x coordinate value of point R so that $r = xR$.

7. The signature can be calculated as follows:

$$S = (h(m) + dr)K_{e^{-1}} \bmod q$$

Here, m is the message for which the signature is being computed, and $h(m)$ is the hash of the message m .

Signature verification is carried out by following this process:

1. Auxiliary value w is calculated as $w = s^{-1} \bmod q$.

2. Auxiliary value $u1 = w \cdot h(m) \bmod q$.

3. Auxiliary value $u2 = w \cdot r \bmod q$.

4. Calculate point P , $P = u1A + u2B$.

5. Verification is carried out as follows: r, s is accepted as a valid signature if the x coordinate of point P calculated in step 4 has the same value as the signature parameter $r \bmod q$; that is, $Xp = r \bmod q$ means a valid signature

$Xp \neq r \bmod q$ means invalid signature
